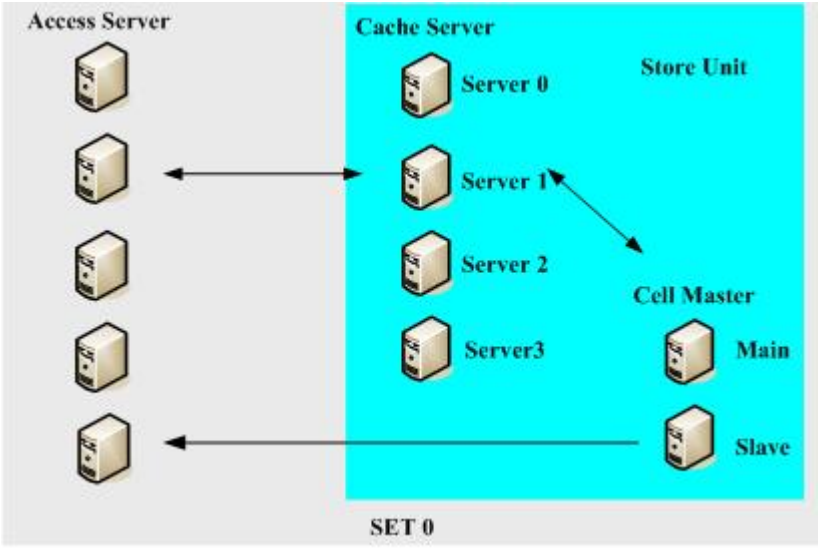
QQ Album & WeChat Multimedia Acceleration Tool — Introduction to TDC

TDC (Tencent Disk Cache) is an SSD-based disk cache system widely adopted by services including QQ Album, WeChat and Mobile QQ Walkie-Talkie. At present, TDC runs on more than 2,000 online servers, deployed in domestic data centers such as DC, OC and AC, as well as overseas sites including the Netherlands, Saudi Arabia, the United States and Singapore. TDC provides large-capacity, high-performance distributed key-value cache services for businesses. It not only reduces the pressure of user requests on back-end DC origin servers but also improves user access experience. This document elaborates on the overall system architecture of TDC and the detailed design of each module.

1. TDC Overall Architecture

TDC consists of two modules: Cell Master and Cache Server. The overall architecture is shown in Figure 1-1.



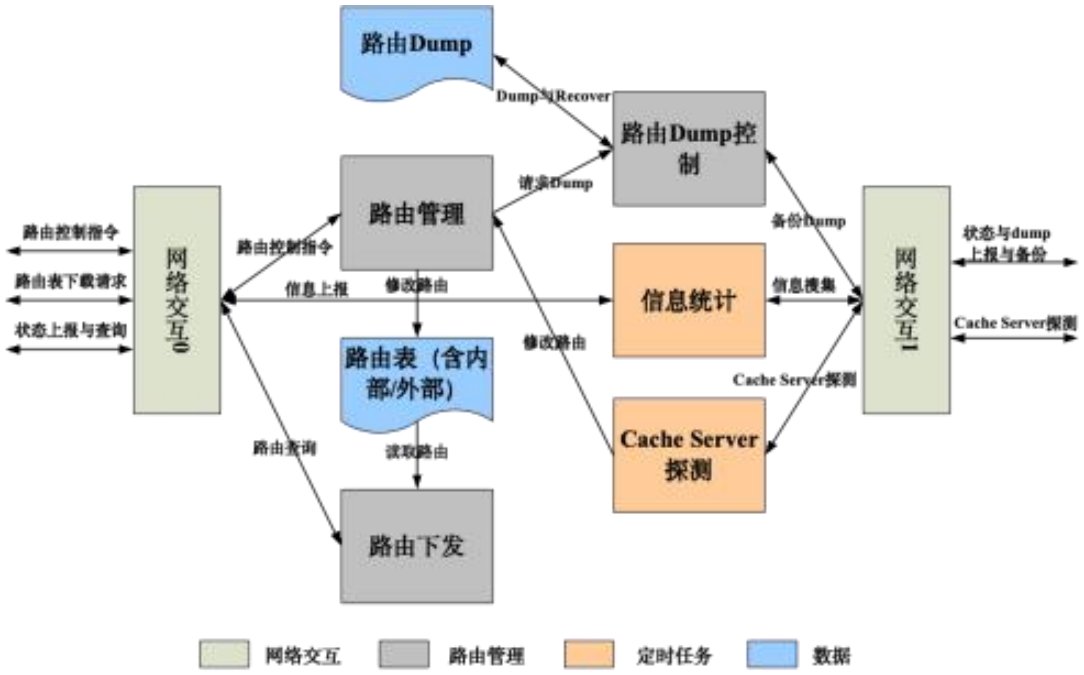
Cell Master, the resource management module, is responsible for managing routing information of the entire system. It periodically detects the running status of each server, automatically removes faulty machines, and receives statistical reports from each server.

Cache Server stores business data. It enables data eviction when disk space usage hits the configured threshold. Its underlying storage engine adopts the `diskcache` module from mcp++. Businesses can choose single-copy or two-copy data storage.

Access Server, the business access module, regularly communicates with Cell Master to obtain up-to-date routing information, and forwards business requests to corresponding Cache Servers according to routing rules.

Implementation of Cell Master

The functional architecture of Cell Master is illustrated in Figure 2-1.



Cell Master mainly comprises three sub-modules: routing management, Cache Server detection & eviction, and Cache Server statistics aggregation.

2.1 Routing Management Module

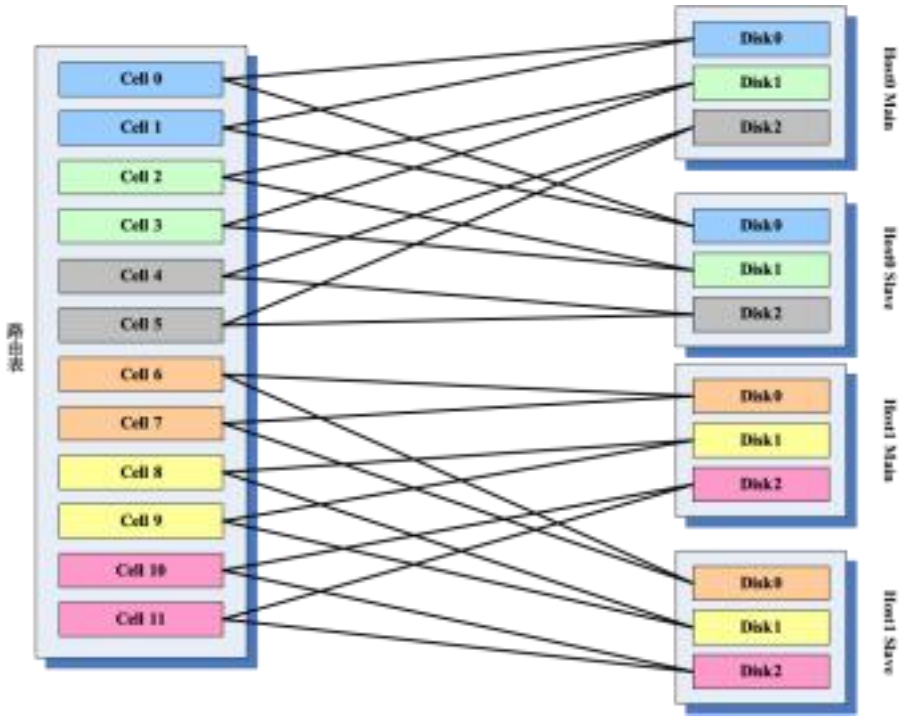
The routing management module maintains and modifies routing information across the whole TDC system. Its key capabilities are listed below: a. Accept requests from business modules and return repository-wide routing information. To save network bandwidth, business modules attach the MD5 value of their local routing table when requesting data. Upon receiving a routing-fetch request, Cell Master compares the MD5 from the business side against its own routing MD5. Updated routing data is returned only when the two MD5 values differ.

b. Externally-triggered routing modification: routing migration for capacity expansion, capacity reduction and faulty machines.

c. Dump routing data: periodically dump routing information and persist it into files.

2.2 Routing Design

The routing logic adopts the classic bucket strategy. The full routing table is divided into 10,000 buckets. The mapping between buckets and disks is one-to-one or many-to-one: one bucket maps to exactly one disk, while a single disk can correspond to one or multiple buckets. The routing principle is shown in Figure 2-2. When a business accesses TDC, it first computes the MD5 of the key, takes modulo 10000 to get the corresponding bucket index, looks up the Cache Server for that bucket in the routing table, and then accesses the target Cache Server.



2.3 Detection and Eviction of Cache Server

Crash detection: Cell Master detects Cache Server liveness via heartbeats. It sends heartbeat protocols to Cache Servers every 15 seconds. If no response is received within 10 seconds, a heartbeat timeout is recorded. A Cache Server is deemed abnormal and evicted after five consecutive heartbeat timeouts.

Bad-disk detection: Bad disks are identified by error-code statistics. Two categories of disk-related error codes are tracked:

1. Write failures when persisting data to disk;
2. Mismatch between on-disk checksum and actual data checksum during data read-out. If a Cache Server reports more than 100 disk errors within one minute, its disk is considered defective, and Master will evict this server.

Eviction mechanism: Crash or fault status updates the routing table on Master. Buckets belonging to the faulty node are marked unavailable. When businesses fetch routing information, the IP for these buckets is set to `255.255.255.255`. Businesses judge node availability by checking this IP value.

2.4 Cache Server Statistics Aggregation

Cache Servers report statistics to Master every minute, and Master aggregates these metrics. Businesses can query Master for repository-level metrics including total stored data volume, per-minute Set/QPS, Get/QPS, Get hit ratio, and disk usage of each Cache Server.

Cache Server

3.1 Cache Server Overall Architecture

Cache Server follows a multi-process plus multi-thread model. One `mcd` process is bound to each SSD drive. Inside each `mcd` process, multiple threads handle business read-write requests. An auxiliary module collects statistics from all `mcd` instances, aggregates metrics for reporting to Master, and responds to Master heartbeat requests. The Cache Server architecture is shown in Figure 3-1.

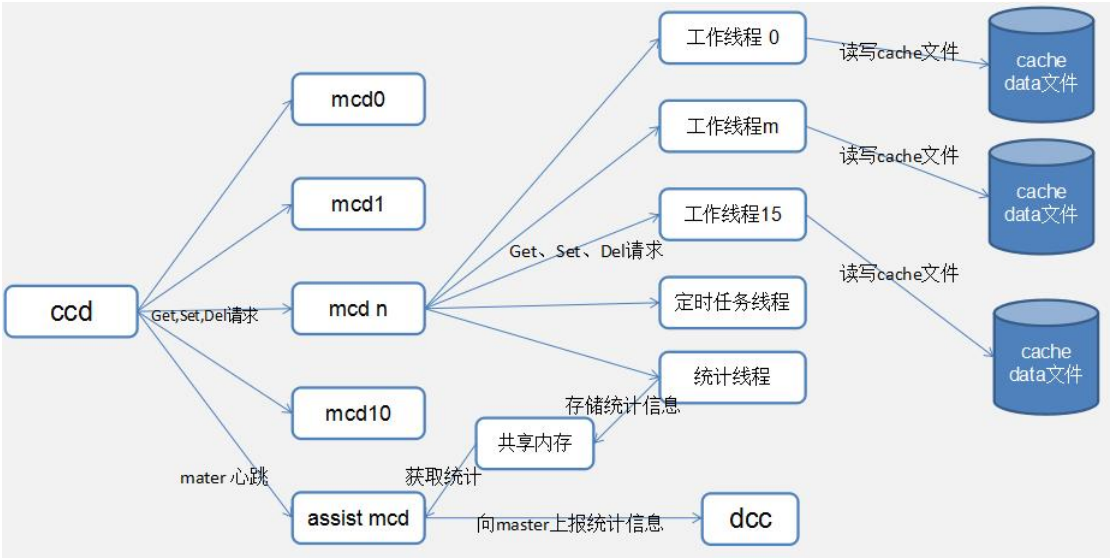


Figure 3-1 demonstrates the process model of Cache Server on a TS8 machine. Roles of each component are described below:

- **ccd**: Receives incoming business requests, dispatches them to corresponding `mcd` processes, and sends responses back to business clients.
- **mcd**: Accepts requests from ccd, dispatches them to internal worker threads, and returns processing results to ccd. One `mcd` process corresponds to one physical disk.
- **assist mcd**: Responds to Master heartbeat requests. It periodically reads statistics from shared memory, aggregates metrics and reports them to Master.
- **Scheduled task thread**: Periodically dumps in-memory metadata of the disk cache component for each worker thread, enabling cache data recovery after process restarts.
- **Statistics thread**: Collects metrics including mcd request volume, error codes, disk cache node occupancy and categories of data inside cache data files, and writes statistics into designated shared memory.
- **cache data file**: File-based cache files storing actual business payloads. Each cache data file is 15 GB in size on TS8 machines.

3.2 Cache Server Storage Engine

The underlying storage engine of Cache Server is the `diskcache` component from mcp++, which manages space for cache data files. DiskCache maintains hash buckets to store keys, together with metadata such as offset, length and checksum of key-value entries inside cache data files. Space allocation for cache data files adopts a mechanism similar to the buddy memory allocation algorithm.
